

Cierre de Sprint – Evidencia Metodológica RUP + SCRUM

1 Información general del Sprint

- **Sprint:** Sprint No. 4
- **Periodo:** 13/Abr/2026 – 26/Abr/2026
- **Objetivo del Sprint:** El objetivo primordial de esta iteración se centró en el refinamiento de la arquitectura de software y la mitigación de riesgos técnicos y sustanciales mediante la maduración de los flujos de trabajo de implementación. Se priorizó la estabilidad técnica a través de la refactorización de la lógica de instanciación hacia un paradigma orientado a vistas y el uso de Prefabs, optimizando así la eficiencia del diseño en tiempo de diseño. Simultáneamente, se fortaleció el valor pedagógico del sistema mediante el alineamiento de cada uno de los Puzzles con los protocolos de bioseguridad, integrando una lógica que consulta la persistencia para mostrar los tutoriales y un sistema de evaluación cuantitativa basado en pesos equilibrados para aciertos y errores. Finalmente, se consolidó la validación con el Product Owner mediante la corrección iterativa de mecánicas de los Puzzles, asegurando la integridad del producto frente a las necesidades del cliente.

Adicionalmente, durante el sprint se desarrolló desde cero el minijuego "Clasifica y Desecha", orientado a la clasificación de residuos de laboratorio, y se mejoró el minijuego "Temperatura Ideal", incorporando validaciones más completas sobre llenado de tubos, método de calentamiento, temperatura ideal, daño del compuesto y feedback visual al jugador.

2 Matriz de Trazabilidad

Fase RUP ↔ Objetivos ↔ Actividades ↔ Artefactos

 Tabla de trazabilidad

Fase RUP	Objetivo de la Fase	Actividades realizadas en el Sprint	Artefactos producidos
Incepción	Definir el alcance del proyecto y establecer la viabilidad de nuevas funcionalidades mediante el acuerdo con los <i>stakeholders</i> .	Consulta y validación con el <i>Product Owner</i> para la aprobación de conceptos de diseño y requisitos de un nuevo sistema de puzzles. Definición del nuevo minijuego de residuos y revisión de mejoras para el minijuego de calentamiento.	Historias de Usuario y tareas relacionadas. Historias de usuario y tareas asociadas al incremento del sprint.
Elaboración	Mitigar riesgos técnicos y establecer una arquitectura base sólida mediante el refinamiento del modelo de diseño.	1. Refactorización del flujo de trabajo hacia una instanciación orientada a vista utilizando el <i>Prefab Mode</i> de Unity. 2. Rediseño del protocolo de limpieza	Línea base de la arquitectura (Modelo de diseño) y Diagramas de clase y de secuencia.

		(BloodStain) para alinearlo con la normativa técnica de bioseguridad. 3. Diseño de reglas para clasificación de residuos y flujo de calentamiento de sustancias.	Reglas funcionales, modelo de interacción y responsabilidades por componente.
Construcción	Desarrollar de forma iterativa e incremental la funcionalidad completa mediante la implementación de componentes de software.	1. Implementación de funciones de consulta de persistencia en el DataManager (Control de tutoriales). 2. Generación de Assets (Sprites) con coherencia narrativa y ajuste de componentes de colisión (<i>Colliders</i>). 3. Codificación de la lógica de evaluación y balanceo de puntaje. 4. Implementación de "Clasifica y Desecha" y mejora funcional de "Temperatura Ideal".	Componentes de código C#, <i>ScriptableObjects</i>, <i>Prefabs</i> configurados y Assets 2D. Scripts C#, <i>ScriptableObjects</i>, <i>prefabs</i> configurados, feedback visual y assets 2D.
Transición	Asegurar que el producto cumple con los estándares de calidad para su entrega al	Ajuste y corrección de los Puzzles "BloodStain" y de calentamiento de	Incremento de software funcional y

	usuario final o validación funcional.	elementos químicos atendiendo las observaciones de calidad y retroalimentación del <i>Product Owner</i>. Pruebas funcionales de interacción, validación, puntaje, finalización y feedback visual.	corrección de <i>bugs</i>. Detalle de entregables técnicos añadidos al sprint: Minijuego "Clasifica y Desecha": ContainerManager.cs, ObjectManager.cs, PuzzleGameplayContainer.cs , disposal.cs, BlankObject.prefab, GamePlayContainers.prefab, PuzzleManagerContainers.prefab, Performance.prefab y TutorialElements.prefab. Minijuego "Temperatura Ideal": ElementManager.cs, TubeManager.c
--	--	---	---

			<code>s, WarmManager.cs, DropZone.cs, GamePlayElements.cs, Element.cs, TuboEnsayo.prefab, BlankElement.prefab, PuzzleElements.prefab, PuzzleManagerElements.prefab y burbujeo_0.prefab.</code> <code>Incremento funcional validado para ambos minijuegos.</code>
--	--	--	---

3 Evidencia de decisiones de diseño por fase

Justificación técnica

En este Sprint se tomaron las siguientes decisiones relevantes:

Decisiones arquitectónicas

- **Decisión:** Uso de Drag & Drop como interacción principal en los minijuegos.
- **Fase RUP asociada:** Elaboración / Construcción.
- **Justificación técnica:** Se mantuvo un sistema de interacción directa mediante arrastre de objetos para facilitar la manipulación de residuos, sustancias y tubos de ensayo dentro del entorno de laboratorio.
- **Impacto en el sistema:** Permite reutilizar lógica de interacción entre minijuegos, reduce complejidad de controles y mejora la comprensión del jugador.
- **Decisión:** Separación de responsabilidades entre objetos, zonas y controladores de gameplay.
- **Fase RUP asociada:** Elaboración / Construcción.
- **Justificación técnica:** Se separó la lógica de residuos, contenedores, sustancias, tubos de ensayo, estaciones de calentamiento y flujo general del puzzle en scripts independientes.
- **Impacto en el sistema:** Mejora la mantenibilidad, permite extender los minijuegos con nuevos residuos o sustancias y reduce el acoplamiento entre componentes.
- **Decisión:** Uso de feedback visual contextual.
- **Fase RUP asociada:** Construcción.
- **Justificación técnica:** Se implementaron mensajes cortos, indicadores visuales y cambios de color para comunicar errores o aciertos sin sobrecargar la interfaz.
- **Impacto en el sistema:** Mejora la claridad de las acciones del jugador y refuerza el aprendizaje mediante retroalimentación inmediata.

Organización del sistema

- **Separación aplicada:**
 - Core
 - Sistema de Puzzles
 - Persistencia
- **Motivo:** Mantener una arquitectura modular en la que el núcleo del juego, los sistemas de puzzles y la persistencia puedan evolucionar sin generar dependencias innecesarias.
- **Beneficio esperado:** Facilitar la reutilización de componentes, mejorar la escalabilidad de nuevos minijuegos y reducir el riesgo de errores al modificar una mecánica específica.

Patrones o principios utilizados

- **Patrón / Principio:** Separación de responsabilidades, reutilización mediante Prefabs, parametrización con ScriptableObjects y composición de componentes en Unity.
- **Contexto de uso:**

Se aplicó en la organización de los minijuegos mediante scripts especializados para objetos, zonas, gameplay, evaluación y feedback. En el minijuego de residuos se separaron los residuos, contenedores y flujo principal; en el minijuego de calentamiento se separaron las sustancias, tubos de ensayo, estaciones de calentamiento y control de temperatura.

- **Fase RUP:** Elaboración / Construcción

4 Bitácora resumida por fase RUP

Incepción

Qué se hizo:

Se revisó el alcance del sprint y los requerimientos necesarios para completar el flujo básico del gameplay del puzzle, identificando los sistemas principales que debían implementarse: salud del jugador, temporizador y condiciones de victoria o derrota.

Además, se identificó la necesidad de incorporar un nuevo minijuego relacionado con la clasificación de residuos de laboratorio y de mejorar el minijuego existente de calentamiento de sustancias. Se definieron los elementos principales de interacción: residuos, contenedores, sustancias, tubos de ensayo, mechero, baño María y zona de entrega.

Por qué se hizo:

Se realizó para asegurar que el sprint estuviera alineado con el objetivo de lograr un flujo funcional completo del minijuego y definir claramente qué componentes debían desarrollarse.

También se hizo para alinear el incremento con las prácticas de bioseguridad y fortalecer el valor pedagógico de los minijuegos mediante interacciones simples y evaluables.

Resultado:

Objetivos del sprint definidos y claridad sobre los sistemas base necesarios para soportar el gameplay del puzzle.

Alcance ampliado con la definición de un minijuego nuevo de clasificación de residuos y un conjunto de mejoras funcionales para el minijuego de calentamiento de sustancias.

Elaboración

Qué se hizo:

Se diseñó la arquitectura base del sistema de puzzles y de los sistemas de salud y tiempo. También se definieron contratos mediante interfaces como IGameOverSubscriber e IDamageable, así como la estructura de la clase abstracta AbstractPuzzleGameplay.

También se diseñaron las reglas funcionales de ambos minijuegos. Para "Clasifica y Desecha" se definieron tres tipos de contenedores: cortopunzantes, biológico y basura general. Para "Temperatura Ideal" se estructuró el flujo de llenado del tubo, calentamiento, validación del método correcto y entrega final.

Por qué se hizo:

Se buscó reducir riesgos técnicos y establecer una arquitectura flexible que permitiera reutilizar código entre distintos puzzles manteniendo la capacidad de personalizar su lógica.

Se realizó para reducir riesgos de interacción y evitar estados inconsistentes, como llenar tubos ocupados, entregar objetos no procesados o activar estaciones de calentamiento con objetos incorrectos.

Resultado:

Base arquitectónica definida mediante interfaces, clases abstractas y diagramas de clase que estructuran el funcionamiento del sistema.

Reglas de interacción y validación definidas para ambos minijuegos, con separación entre lógica de gameplay, estado de objetos, zonas de soltado y feedback visual.

Construcción

Qué se hizo:

Se implementaron los sistemas funcionales del juego, incluyendo la lógica y vista del sistema de salud (PlayerHealthLogic y PlayerHealthView), el temporizador (CountdownBarManager y CountdownTextManager), así como la lógica de victoria y derrota dentro del flujo del puzzle.

Se desarrolló desde cero el minijuego "Clasifica y Desecha", implementando los scripts ContainerManager, ObjectManager, PuzzleGameplayContainer y disposal, junto con los prefabs necesarios para residuos, contenedores, feedback y tutorial.

También se mejoró el minijuego "Temperatura Ideal", ajustando los scripts ElementManager, TubeManager, WarmManager, DropZone y GameplayElements para permitir llenado de tubos, calentamiento por método correcto, control de temperatura, daño del compuesto, feedback contextual y cálculo de puntaje.

Por qué se hizo:

Se desarrollaron estos componentes para materializar la arquitectura diseñada y permitir que el jugador interactúe con el puzzle dentro de un flujo completo de juego.

Estas implementaciones permitieron convertir las reglas diseñadas en mecánicas jugables, medibles por puntaje y coherentes con el contexto de laboratorio universitario.

Resultado:

Prototipo funcional del gameplay con sistemas de salud, temporizador y transición entre estados de victoria o derrota.

Incremento funcional con un minijuego nuevo de clasificación de residuos y una versión mejorada del minijuego de calentamiento de sustancias, ambos integrados con interacción Drag & Drop, validación, feedback y evaluación.

Transición

Qué se hizo:

Se realizaron pruebas de integración en una escena de prueba verificando el flujo completo del juego: entrada al puzzle, tutorial, desarrollo del gameplay y retorno al juego principal tras ganar o perder.

Adicionalmente, se realizaron pruebas funcionales verificando que los residuos se clasificaran correctamente, que los tubos de ensayo siguieran el flujo esperado y que las acciones incorrectas mostraran feedback adecuado.

Por qué se hizo:

Se evaluó el funcionamiento conjunto de los sistemas implementados para validar que el flujo del jugador fuera consistente y que las transiciones entre estados se ejecutaran correctamente.

Se buscó comprobar que las mejoras no generaran errores de flujo, estados inválidos o confusión para el jugador durante la interacción con objetos y zonas.

Resultado:

Comprobación del flujo funcional del puzzle dentro de una escena de prueba, confirmando la correcta integración de los sistemas desarrollados.

Validación del comportamiento funcional de ambos minijuegos, confirmando la correcta finalización del puzzle, el cálculo de puntaje y la coherencia del feedback visual.

5 Validación del incremento (si aplica)

- **Funcionalidad:** OK – Se implementó completamente el minijuego "Clasifica y Desecha" y se mejoró el flujo funcional del minijuego "Temperatura Ideal".
- **Fiabilidad:** OK – Los minijuegos responden correctamente ante acciones válidas e inválidas del jugador, evitando estados inconsistentes como llenar tubos ocupados o entregar elementos no procesados.
- **Rendimiento:** OK – El uso de prefabs, ScriptableObjects y generación controlada de objetos permite mantener una ejecución estable durante el gameplay.
- **Usabilidad:** OK – El feedback visual, los mensajes cortos y los indicadores de estado permiten al jugador comprender aciertos, errores y progreso dentro de cada minijuego.